

Protective Computing: An Architectural Framework for Safety-Critical Systems Under Conditions of Human Vulnerability

K. OVERTON, Independent Researcher, Canada

The Stability Assumption—that users enjoy continuous connectivity, cognitive surplus, environmental safety, and institutional trust—has silently governed software architecture for three decades. This paper argues that this assumption is structurally incompatible with the material conditions of the 2020s, where a growing proportion of human-computer interaction occurs under vulnerability: medical crisis, environmental disaster, coercion, and precarity. We introduce Protective Computing, a systems-engineering orientation that inverts conventional priorities, mandating safety, containment, reversibility, and utility over growth and engagement. The framework formalizes Stability Bias as a function of dependency, vulnerability, and irreversibility ($H = f(D, V, I)$); defines a Vulnerability State Machine (S_0 – S_3); and specifies five normative design principles using RFC-style keywords. We demonstrate technical feasibility through a reference implementation (PainTracker), yielding a Protective Legitimacy Score in the protective range (presented as an illustrative calculation; independent audit pending). We conclude with implications for HCI, digital medicine, and law, arguing that software architecture has become a determinant of human safety and requires corresponding evaluation standards.

CCS Concepts: • **Human-centered computing** → HCI theory, concepts and models; • **Networks** → Network reliability; • **Security and privacy** → Human and societal aspects of security and privacy.

Additional Key Words and Phrases: Protective Computing; Stability Bias; Vulnerability States; Local-First Architecture; Client-Side Encryption; Digital Iatrogenesis; Human-Computer Interaction; Safety-Critical Systems; Coercion Resistance; Data Sovereignty

ACM Reference Format:

K. Overton. 2026. Protective Computing: An Architectural Framework for Safety-Critical Systems Under Conditions of Human Vulnerability. 1, 1 (February 2026), 15 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Consider a patient in acute pain attempting to verify medication history through a mobile application. The application requires network connectivity to authenticate—but the hospital’s Wi-Fi is down. Consider a survivor of domestic abuse whose device is monitored by a hostile partner; a lock-screen notification reveals appointment details with a shelter. Consider a displaced person after a flood, with intermittent power and connectivity, attempting to access digital identification to receive aid.

In each case, software designed for stable conditions fails not gracefully, but harmfully. These are not edge cases. They are structurally recurrent states of human experience that reveal a fundamental flaw in the architectural logic of modern computing.

Author’s Contact Information: K. Overton, Independent Researcher, Kelowna, BC, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/2-ART

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1.1 The Stability Assumption

The foundational logic of modern software architecture emerged between 1995 and 2015, a period of exceptional infrastructural optimism. Driven by commercial imperatives, engineering standards converged around three priorities: connectivity, synchronization, and data accumulation. Over time, these priorities hardened into a silent but pervasive architectural doctrine: the Stability Assumption.

This assumption presumes a default user operating within a bounded envelope defined by four conditions:

- **Continuous connectivity:** reliable, high-speed network access
- **Cognitive surplus:** sufficient executive function to manage permissions and authentication
- **Environmental safety:** physical security free from surveillance or coercion
- **Institutional trust:** a working belief that platforms function as neutral intermediaries

For nearly three decades, these conditions have been treated as constants. When systems fail outside this envelope, the failure is rarely attributed to architecture. The system is described as "offline," the user as "non-compliant."

1.2 The Legitimacy Gap

This stability-centric model is increasingly incompatible with the material conditions of human life. A growing proportion of human-computer interaction now occurs outside the Stability Envelope: medical crisis, environmental disaster, coercive environments, socioeconomic precarity.

When stability-centric systems encounter vulnerability, they do not merely degrade—they amplify harm. A network-dependent smart lock that cannot disengage during a power outage traps occupants. A cloud-first medical journal that denies access when a user cannot recall credentials during an emergency creates clinical risk.

This divergence constitutes a Legitimacy Gap: software that requires stability to function becomes ethically illegitimate when applied to domains of health, safety, or legal survival. We propose Protective Computing as a corrective discipline—an engineering orientation in which the system's primary obligation is the preservation of agency, dignity, and safety under conditions of collapse.

1.3 Contributions

This paper makes six contributions:

- (1) A formal model of Stability Bias and the Vulnerability State Machine
- (2) A taxonomy of protective failures based on human harm, not technical cause
- (3) Five normative design principles for Protective Computing, specified with RFC-style keywords
- (4) A reference implementation (PainTracker) demonstrating technical feasibility
- (5) The Protective Legitimacy Score (PLS), a proposed metric for comparative evaluation (provisional and illustrative)
- (6) Implications for HCI, digital medicine, and law, with a research agenda for the field

2 Related Work

Any proposal for a new engineering orientation must locate itself within existing intellectual traditions. Protective Computing emerges from, and synthesizes, four established lines of inquiry while advancing a distinct claim that reorganizes them. This section situates the framework within safety-critical HCI, value-sensitive design and privacy-by-design, resilience engineering and local-first architecture, and technology-facilitated abuse research. We then articulate the delta: where prior work treats vulnerability as a special case, Protective Computing treats it as the default condition for evaluation.

2.1 Safety-Critical HCI

Human-Computer Interaction has long recognized safety as a design constraint in domains where failure produces catastrophic consequences. Foundational work in aviation cockpit automation established that interface design must anticipate and mitigate operator error under stress [24, 32]. Nuclear control room studies demonstrated that system transparency and error recovery mechanisms are not optional but essential for preventing disaster [34]. Healthcare HCI has extended these insights to clinical settings, showing that medication administration systems and electronic health records must function reliably even when users are fatigued, distracted, or under time pressure [17, 23].

The core insight of this tradition—that interface design must accommodate degraded human performance—informs our Principle 4 (Cognitive Load Preservation). However, safety-critical HCI has historically focused on trained operators in controlled environments: pilots with thousands of hours of experience, nuclear technicians who undergo regular simulation training, clinicians working within institutional protocols. These are not the general population under conditions of precarity, coercion, or infrastructural collapse.

Protective Computing extends safety-critical HCI in three ways. First, it addresses untrained users who cannot be assumed to possess domain expertise. Second, it considers environments where the infrastructure itself is failing, not merely the operator. Third, it formalizes vulnerability as a state variable rather than a static characteristic, enabling systems to adapt as users transition between conditions. In this sense, Protective Computing applies the rigor of safety-critical engineering to contexts that have historically been served by consumer-grade software.

2.2 Value-Sensitive Design and Privacy-by-Design

Value-sensitive design (VSD) provides methodologies for embedding ethical considerations into technical systems from the outset. Friedman and colleagues established that values can be operationalized as system requirements through iterative investigation of human contexts, conceptual analysis of value tensions, and technical implementation of value-based constraints [7, 8]. Subsequent work has applied VSD to domains including privacy in ubiquitous computing [28], informed consent in data collection [21], and autonomy in assistive technologies [31].

Privacy-by-design, articulated by Cavoukian and extended by regulatory frameworks including the GDPR, translates similar commitments into engineering practice [1, 4]. The seven foundational principles—proactive not reactive, privacy as default, privacy embedded into design, full functionality, end-to-end security, visibility and transparency, and user-centricity—have influenced everything from system architecture to data protection law [26]. More recent work has examined the limits of privacy-by-design in contexts of asymmetric power, arguing that consent mechanisms fail when users cannot meaningfully refuse [10, 25].

Protective Computing adopts the VSD stance that values can be operationalized as system requirements, while specifying a particular value hierarchy: safety and sovereignty outweigh convenience and engagement. Where VSD asks which values to embed and how to balance competing values, Protective Computing argues that vulnerability fundamentally reorders value priorities. A system optimized for convenience in stable conditions becomes harmful in crisis; the hierarchy must invert. This is not a rejection of VSD but a context-specific instantiation of its methodology.

Similarly, privacy-by-design establishes that systems should minimize data collection and empower users. Protective Computing extends this by mandating client-side encryption and local-first authority, transforming privacy from a compliance exercise into a structural guarantee. The framework also addresses a gap in privacy-by-design: the assumption that users can exercise meaningful

choice. Under vulnerability states (S_2 , S_3), this assumption fails, and systems must protect users from their own compromised agency.

2.3 Resilience Engineering and Local-First Architecture

Resilience engineering studies how systems maintain function under disruption. Hollnagel and colleagues distinguished Safety-I (preventing things from going wrong) from Safety-II (ensuring things go right), arguing that resilience requires systems to adapt to unexpected conditions rather than merely resist failure [12, 13]. Woods extended this by defining resilience as the capacity to anticipate, monitor, respond to, and learn from disruptions [33]. In safety-critical domains, resilience is not a performance characteristic but a survival requirement.

Parallel technical movements have demonstrated that network independence is architecturally achievable. Offline-first and local-first software architectures prioritize local authority over cloud dependency, using techniques including embedded databases, CRDTs (Conflict-Free Replicated Data Types), and peer-to-peer synchronization [14, 22, 27]. Kleppmann and colleagues articulated the local-first manifesto: users own their data, network failures do not block work, and synchronization occurs transparently when connectivity permits [16]. These approaches have been implemented in production systems ranging from collaborative editors to personal information management tools [9, 11].

Protective Computing unifies resilience engineering and local-first architecture through an ethical lens. Resilience is not merely desirable but morally obligatory when failure produces human harm. Local-first architecture is not merely a technical preference but a structural requirement for systems operating in vulnerability contexts. Where prior work treated resilience as a performance metric and local-first as a design pattern, Protective Computing treats them as duties owed to users who cannot afford network dependency. The framework provides the normative justification for adopting these technical approaches in safety-critical domains.

2.4 Technology-Facilitated Abuse and Digital Safety Research

A growing body of work examines how digital systems enable coercion, stalking, and surveillance. Freed and colleagues conducted qualitative studies with survivors of intimate partner violence, documenting how smartphones and connected devices become tools of abuse [5, 6]. Matthews and colleagues analyzed how "stalking apps" exploit operating system permissions and cloud synchronization to monitor victims without consent [20]. Levy and colleagues examined the legal frameworks that fail to protect survivors when technology companies resist disclosure or design systems that preserve evidence of abuse [19, 30].

This research documents the failure modes we classify as Type II (Involuntary Exposure) and Type IV (Coercive Compliance). Lock-screen notifications that reveal sensitive appointments, location sharing that cannot be disabled without detection, authentication mechanisms that assume physical safety—these are not edge cases but recurring harms documented across multiple studies [2, 3]. Digital safety researchers have called for architectural interventions, arguing that privacy and security cannot be achieved through user education alone when systems are designed for surveillance by default [29].

Protective Computing provides an architectural framework for addressing these harms systematically. Rather than treating technology-facilitated abuse as a problem to be mitigated through post-hoc patches or user warnings, the framework embeds coercion resistance into the system's foundation. Duress codes, plausible deniability storage, and local-first authority are not features added after the fact but structural guarantees that anticipate adversarial conditions. The framework translates empirical findings from digital safety research into engineering requirements, closing the gap between documentation of harm and prevention of harm.

2.5 The Delta: Vulnerability as Default

Where prior work treats vulnerability as a special case—a scenario to be accommodated within systems designed for stability—Protective Computing inverts the default. This inversion is not merely semantic but structural: it reorganizes design priorities, evaluation criteria, and architectural choices.

Safety-critical HCI assumes trained operators in controlled environments. Value-sensitive design asks which values to embed but does not specify a hierarchy for vulnerability. Resilience engineering treats adaptation as a performance characteristic. Local-first architecture demonstrates technical feasibility without ethical mandate. Technology-facilitated abuse research documents harm without providing architectural solutions.

Protective Computing synthesizes these traditions while advancing a distinct claim: vulnerability is not an edge condition but a structurally recurrent human state, and architecture must be evaluated primarily by its behavior when stability degrades. This yields a new organizing principle: software’s legitimacy in safety-critical domains is determined not by its performance under ideal conditions, but by its capacity to preserve agency and dignity when those conditions fail.

The framework does not claim to supersede these traditions. Rather, it builds upon them, extracting insights from each and integrating them into a coherent orientation for designing under vulnerability. Safety-critical HCI provides the rigor; VSD provides the methodology; resilience engineering provides the systems perspective; local-first architecture provides the technical patterns; digital safety research provides the empirical grounding. Protective Computing is the synthesis that makes these traditions legible as a unified engineering discipline.

3 Formal Model: Stability Bias and Vulnerability States

3.1 Systemic Definition of Harm

In safety-critical engineering, risk is traditionally expressed as the product of probability and consequence. Protective Computing extends this formulation by incorporating the dynamic state of the user as a primary variable. We model Total Harm (H) in a digital interaction as:

$$H = f(D, V, I)$$

Where:

- **D — System Dependency:** The degree to which the user relies on the system for immediate biological survival, legal protection, or preservation of irreplaceable data
- **V — User Vulnerability:** The momentary reduction in user agency caused by degradation of environmental stability, cognitive capacity, or power autonomy
- **I — Outcome Irreversibility:** The permanence of a negative system outcome (e.g., transient lockout versus irreversible data destruction)

The functional form is intentionally abstract; operationalization of D , V , and I requires domain-specific calibration and empirical study. Different application contexts will require different measurement approaches, and the relationships between these variables may be non-linear. This formulation is offered as a conceptual framework for analysis rather than a predictive equation.

3.2 Formal Definition: Stability Bias

Stability Bias exists when an architecture exhibits high D and high I while implicitly assuming $V \rightarrow 0$. Under this condition, any sudden increase in V produces catastrophic escalation of H , transforming routine system failure into human harm.

3.3 The Four Structural Dependencies of Stability

Stability Bias is embedded in software architecture through four dependency vectors:

Table 1. Structural Dependencies of Stability

Dependency	Description
Infrastructure	Reliance on continuous, low-latency network connectivity for authentication, synchronization
Cognitive	Requirement for sustained executive function to manage permissions, navigation hierarch
Environmental	Assumption of physical privacy and absence of hostile observation during interaction
Resource	Assumption of modern hardware capability, stable electrical power, and unmetered data b

Failure across any vector increases V , thereby amplifying H in stability-biased systems.

3.4 The Vulnerability State Machine

Traditional HCI relies on static personas. Such representations are incompatible with protective design because vulnerability is temporal, not categorical. Protective Computing replaces personas with state-based modeling, representing the user as an agent transitioning within a Vulnerability State Machine (VSM).

Table 2. Vulnerability State Machine

State	Designation	Conditions
S_0	Stable	Reliable connectivity, cognitive clarity, environmental safety
S_1	Degraded	Intermittent connectivity, fatigue, moderate stress, public exposure
S_2	Acute Crisis	Connectivity loss, severe pain or panic, imminent physical or psychological threat
S_3	Coercive	Device inspection or control by hostile third party

Unidirectional Ratchet Principle: Protective systems must implement asymmetric transition cost—transition downward into safer states must be effortless; transition upward into more vulnerable states must require deliberate, verifiable confirmation.

3.5 Taxonomy of Protective Failures

Protective Computing classifies failure by human consequence rather than technical cause:

Table 3. Taxonomy of Protective Failures

Failure Class	Definition
Type I: Denial of Self	System blocks user access to locally owned data due to remote dependency
Type II: Involuntary Exposure	System reveals sensitive data without valid contemporaneous consent
Type III: Irreversible Erasure	System permits permanent destruction of critical data during vulnerability
Type IV: Coercive Compliance	System demands action user cannot safely perform

This taxonomy reframes failure as ethical severity, not merely operational defect.

4 The Five Protective Design Principles

To qualify as Protective Architecture, a system **MUST** implement the following constraints. Normative keywords (**MUST**, **MUST NOT**, **SHOULD**, **MAY**) are interpreted according to RFC 2119 [15] and RFC 8174 [18].

4.1 Principle 1: Radical Reversibility

Formal Definition: A protective system **MUST** guarantee that no user action executed within a detected Vulnerability State ($V > 0$) results in immediate or irreversible loss of user data.

Threat Model: A user in cognitive distress (S_2) or coercion (S_3) executes a destructive command due to panic, confusion, or external pressure.

Engineering Constraints:

- **Immutable Append-Only Logging:** The persistence layer **MUST** implement append-only semantics. Logical deletion **MUST** be represented as a non-destructive cryptographic tombstone.
- **Restoration Window:** All destructive state transitions **MUST** remain locally reversible for a minimum configurable duration (RECOMMENDED: ≥ 72 hours) without administrative or network dependency.
- **Destructive Friction:** High-impact operations **MUST** require deliberate, non-trivial confirmation **EXCEPT** when the system detects imminent physical threat conditions.

Architectural Pattern: Soft-Deletion Protocol — records marked `deleted=true` are excluded from default queries yet retained in encrypted storage until explicit, user-authorized garbage collection occurs.

4.2 Principle 2: Minimum Necessary Exposure (Zero-Knowledge Default)

Formal Definition: A protective system **MUST** minimize remote data exposure to the absolute minimum required for local functionality. All remote infrastructure **MUST** be treated as an untrusted adversarial environment.

Threat Model: Institutional compromise (legal seizure, breach, or insider threat) exposes centrally stored user data.

Engineering Constraints:

- **Client-Side Encryption:** All user-generated content **MUST** be encrypted prior to transmission. Servers **MUST NOT** possess decryption capability.
- **Metadata Minimization:** Ambient telemetry collection **MUST NOT** occur unless strictly required for an explicit, user-initiated synchronous function.
- **Ephemeral Session Semantics:** Sensitive authentication tokens **MUST** maintain short TTL and **MUST NOT** persist across reboot without re-verification.

Architectural Pattern: Blind Server Model — the backend functions solely as a relay for encrypted payloads without semantic visibility.

4.3 Principle 3: Failure Containment (Local-First Authority)

Formal Definition: A protective system **MUST** preserve full read/write capability and data integrity independent of external infrastructure status.

Threat Model: Network, power, or server failure occurs during a safety-critical user interaction.

Engineering Constraints:

- **Local-First System of Record:** The local database **MUST** be authoritative. Network services **MUST NOT** be required for core functionality.

- **Non-Destructive Conflict Resolution:** Synchronization conflicts MUST preserve all divergent states (e.g., CRDTs or multi-version retention) pending user adjudication.
- **Optimistic Interface Semantics:** UI state transitions MUST reflect local success immediately, remaining decoupled from network latency or availability.

4.4 Principle 4: Cognitive Load Preservation

Formal Definition: Interfaces operating within Vulnerability States (S_2 , S_3) MUST minimize executive function demand and default to heuristic simplicity.

Threat Model: Cognitive overload prevents task completion during crisis conditions.

Engineering Constraints:

- **Deterministic Crisis Interface Routing:** The view layer MUST support state-dependent rendering characterized by: $\geq 50\%$ reduction in information density; enlarged touch targets and increased contrast; elimination of deep or non-linear navigation paths.
- **Persistent Interaction State:** Application context MUST survive crashes, termination, and reboot, restoring exact pre-interruption state.
- **Interrupt Suppression:** Non-critical notifications, prompts, or marketing modals MUST NOT appear during S_2 or S_3 .

Architectural Pattern: Context-Aware View Controller — global state management routes interaction through vulnerability-specific UI trees.

4.5 Principle 5: Asymmetric Power Defense

Formal Definition: Protective systems MUST actively mitigate coercion, surveillance exposure, and vendor lock-in.

Threat Model: Forced device access by hostile actors or institutional restriction of data portability.

Engineering Constraints:

- **Coercion-Resistant Authentication:** Systems MUST support alternate credentials ("Duress Codes") that trigger protective behaviors: hiding sensitive partitions, presenting neutral data, clearing volatile memory.
- **Universal Offline Exportability:** Complete user data export in non-proprietary formats (JSON/CSV) MUST be available without server interaction.
- **Plausible Deniability Storage:** Hidden-volume or steganographic storage SHOULD be implemented where platform capabilities permit.

Architectural Pattern: Panic Protocol — a rapid, user-triggerable mechanism executes predefined safety routines (encrypted volume unmount, state concealment).

5 Reference Implementation: PainTracker

To determine whether Protective Computing is computationally realizable rather than purely theoretical, we constructed PainTracker as a reference implementation according to the Sovereign-Local Stack defined above.

5.1 Structural Alignment

This alignment demonstrates full conformance with the Sovereign-Local constraint model.

Table 4. Structural Alignment: Stability-Centric vs. Protective Architecture

Architectural Dimension	Stability-Centric SaaS	Protective Reference Implementation
System of Record	Remote relational database	Embedded local database
Identity Authority	Server-validated session	Local cryptographic derivation
Operational Dependency	Network-required	Offline-first
Confidentiality Model	Server-side encryption keys	Client-side encryption keys
Data Portability	Administrative export workflow	Immediate offline structured export

5.2 Verification of Radical Reversibility

Observed Mechanism: Symptom entries are stored within an append-only cryptographic log. Deletion events generate tombstone markers while preserving encrypted payloads for a rolling restoration interval (≥ 72 hours).

Protective Effect: Erroneous deletion under cognitive impairment (S_2) becomes locally reversible without administrative mediation, neutralizing Type III (Irreversible Erasure) harm.

5.3 Verification of Cognitive Load Preservation

Observed Mechanism: Activation of a deterministic Flare Mode suspends analytic dashboards, restricts the interface to two binary safety actions, and increases contrast, scale, and navigational linearity.

Protective Effect: Critical interaction remains executable under severely reduced executive function, preserving task completion capability during high-pain states.

5.4 Verification of Asymmetric Power Defense

Observed Mechanism: Optional synchronization transmits only AES-256 encrypted payloads. Keys are derived from a user passphrase using Argon2id; where secure hardware is available, derived keys are stored and used within that environment. No keys are ever transmitted.

Protective Effect: Compromise, subpoena, or seizure of remote infrastructure yields computationally inaccessible user data, minimizing the Sovereignty Gap (ΔP) and neutralizing Type II (Involuntary Exposure) harm.

5.5 Observed Performance Baselines

Evaluation within high-vulnerability user contexts produced:

- Availability: 100% under network absence
- Local read/write latency: < 50 ms
- Sovereignty Score: 1.0 (complete local cryptographic authority)

These observations derive from controlled testing in development and pilot contexts with approximately two dozen participants over a six-month period. They do not constitute large-scale deployment validation but demonstrate technical feasibility under realistic usage conditions.

5.6 Empirical Conclusion

The reference artifact demonstrates that: (1) Failure Containment is technically achievable; (2) Local Sovereignty is operationally sustainable; (3) Protective Computing is implementable within contemporary consumer hardware constraints. The framework thus transitions from normative specification to empirically supported architecture.

6 The Protective Legitimacy Score (PLS)

To enable comparative evaluation and potential certification, we introduce a candidate metric: the Protective Legitimacy Score (PLS). This composite index is intended as a starting point for discussion and refinement; its weights and submetrics are provisional and may be adjusted per domain. The PLS is presented as an illustrative proposal, not a validated instrument.

6.1 Component Metrics

The PLS aggregates five factors aligned with the five Protective Design Principles, each normalized to [0, 1]:

Table 5. Protective Legitimacy Score Components

Metric	Symbol	Principle	Default Weight
Reversibility Quotient	RQ	Radical Reversibility	20%
Exposure Ratio	ER	Minimum Necessary Exposure	20%
Local Authority Index	LAI	Failure Containment	25%
Cognitive Load Factor	CLF	Cognitive Load Preservation	15%
Sovereignty Delta	ΔS	Asymmetric Power Defense	20%

Weights are illustrative; domain-specific profiles (e.g., clinical vs. legal vs. consumer) may assign different priorities. The selection and weighting of these components have not been empirically validated and are offered to provoke community discussion.

6.2 Formal Calculation

$$PLS = 0.2(RQ) + 0.2(1 - ER) + 0.25(LAI) + 0.15(CLF) + 0.2(1 - \Delta S)$$

6.3 Provisional Interpretation

- $PLS \geq 0.85$: Candidate for Protective Architecture certification after third-party audit
- $0.60 \leq PLS < 0.85$: May be acceptable with explicit user warnings and context-specific risk assessment
- $PLS < 0.60$: Likely unsuitable for safety-critical domains

These thresholds are based on preliminary analysis and require validation through empirical studies and regulatory consultation. They should not be interpreted as established standards.

6.4 Illustrative Calculation: PainTracker (Hypothetical Example)

To illustrate how the PLS might be calculated, we apply it to the PainTracker reference implementation. The following values are estimates based on the implementation described in Section 5; they have not been independently verified and are presented solely to demonstrate the calculation method.

$$PLS = 0.2(0.95) + 0.2(0.95) + 0.25(1.00) + 0.15(0.90) + 0.2(0.90) = 0.945$$

This hypothetical calculation suggests that PainTracker would meet the criteria for Protective Architecture if independently audited. The calculation is presented for illustrative purposes only and should not be interpreted as a claim of certification.

Table 6. Illustrative PLS Calculation for PainTracker

Metric	Estimated Value	Normalized Score
RQ	0.95	0.95
ER	0.05	$1 - 0.05 = 0.95$
LAI	1.00	1.00
CLF	0.90	0.90
ΔS	0.10	$1 - 0.10 = 0.90$

6.5 Limitations and Future Work

Several important limitations must be acknowledged. The PLS weights and component definitions have not been empirically validated; they reflect the author’s judgment and are offered to stimulate discussion. Inter-rater reliability has not been established; different evaluators might assign different scores to the same system. The score does not capture all possible vulnerabilities (e.g., physical side-channel attacks, social engineering, or threats to device integrity). A formal certification would require a third-party audit framework with standardized protocols, trained assessors, and validated instruments.

The PLS is presented as a starting point for community engagement, not as a finished metric. We invite critique, refinement, and empirical validation through future research. The goal is to develop evaluation methods that can inform procurement decisions, regulatory standards, and design practice—but this will require collective effort.

7 Implications

7.1 For Human-Computer Interaction: The End of "Delight"

For two decades, HCI methodology has prioritized metrics of "delight," "engagement," and "frictionlessness." While appropriate for entertainment, these metrics are unsuited for critical infrastructure. HCI must transition toward safety-critical design, adopting methodologies historically reserved for aviation, nuclear, and medical interfaces. A valid interface is not defined by retention or time-on-site, but by safe exit—the capacity to disengage without harm is co-equal in importance to the capacity to engage. This suggests new evaluation criteria for systems operating in vulnerability contexts: time-to-critical-action, error recovery under stress, and graceful degradation should supplement or replace engagement metrics.

7.2 For Digital Medicine: Recognizing Digital Iatrogenesis

In clinical practice, iatrogenesis refers to harm caused by the healer. We formally define Digital Iatrogenesis as harm caused by the rigid architecture of a health application (e.g., a suicide prevention tool mandating password reset during a crisis, a pain tracking application denying access when authentication servers are unreachable). Regulatory bodies including the FDA, Health Canada, and EMA should consider expanding "Software as a Medical Device" (SaMD) classification to include architectural safety. Applications indicated for chronic pain, mental health, or postoperative care may require Protective Audit certification. Failure to provide offline access or local sovereignty could be considered a contraindication for clinical use.

7.3 For Law: The Doctrine of Digital Duress

Contract law relies on the "meeting of the minds." No such meeting occurs when one party is an algorithm and the other is a human in a Vulnerability State (S_2 , S_3). Terms of Service agreements

or data waivers executed during a detected Crisis State should be legally voidable. Data Duress becomes a defensible legal concept: a user cannot validly consent to surveillance, liability waivers, or data extraction while seeking emergency aid. This has implications for evidentiary standards, consumer protection law, and platform liability.

8 Research Agenda and Grand Challenges

The framework establishes axioms, but significant technical barriers remain. We identify three Grand Challenges for the computer science community.

8.1 Challenge I: The Lost Key Paradox (Cryptographic Usability)

Problem: Protective Computing mandates Client-Side Encryption (Principle 2). However, users in trauma states (S_2) frequently experience memory deficits. Key loss results in catastrophic data loss (Type I Harm). This creates a tension between security and accessibility that cannot be resolved through conventional password management.

Research Target: Development of Social Recovery Protocols that enable key restoration via trusted human contacts (e.g., Shamir's Secret Sharing variants) without exposing keys to central servers or requiring complex technical coordination. Threshold cryptography combined with out-of-band verification may offer paths forward, but usability under stress remains unproven.

8.2 Challenge II: Biometric Stress Detection Without Surveillance

Problem: To trigger Crisis Mode automatically, the system requires biometric inference (detecting tremors, erratic input, or interaction patterns indicative of distress). Transmitting this telemetry to the cloud constitutes a privacy violation and creates new surveillance vectors.

Research Target: Optimization of Local-Only Inference Models (TinyML). Algorithms must run entirely on-device to detect stress patterns, ensuring the device understands user state while the vendor remains blind. This requires advances in model compression, energy-efficient inference, and privacy-preserving sensor processing.

8.3 Challenge III: Offline AI for Vulnerable Contexts

Problem: Current AI systems (LLMs) rely on massive cloud connectivity. Users in shelters or disaster zones cannot access legal or medical synthesis without exposing queries to corporate surveillance. This creates a dependency that violates Principle 2 (Minimum Necessary Exposure).

Research Target: Optimization of Small Language Models (SLMs) capable of running on consumer hardware to provide legal/medical synthesis offline. Intelligence must become a local utility, not a cloud subscription. This requires advances in model distillation, on-device inference, and domain-specific compression.

9 Conclusion: From Convenience to Safety

This paper has argued that the Stability Assumption—the unexamined architectural doctrine that users enjoy continuous connectivity, cognitive surplus, environmental safety, and institutional trust—has silently governed software design for three decades. We have demonstrated that this assumption is structurally incompatible with the material conditions of the 2020s, where human-computer interaction increasingly occurs under vulnerability: medical crisis, environmental disaster, coercion, and precarity.

9.1 Summary of Contributions

We introduced Protective Computing as a systems-engineering orientation that inverts conventional priorities, mandating safety, containment, reversibility, and utility over growth and engagement. The framework makes four primary contributions to the literature.

First, we formalized **Stability Bias** as a function of dependency, vulnerability, and irreversibility ($H = f(D, V, I)$), providing a mathematical basis for analyzing how architectural choices translate into human harm. This formalization extends safety-critical engineering by incorporating the dynamic state of the user as a primary variable.

Second, we defined a **Vulnerability State Machine** (S_0-S_3) that replaces static personas with state-based modeling, recognizing that vulnerability is temporal rather than categorical. This enables systems to adapt their behavior as users transition between stable, degraded, acute crisis, and coercive states.

Third, we specified **five normative design principles** using RFC-style keywords (Radical Reversibility, Minimum Necessary Exposure, Failure Containment, Cognitive Load Preservation, Asymmetric Power Defense). These principles translate ethical commitments into engineering constraints, providing actionable guidance for system architects.

Fourth, we demonstrated technical feasibility through a **reference implementation** (PainTracker), showing that Protective Computing is realizable within contemporary consumer hardware constraints. The implementation achieved 100% availability under network absence in controlled testing and complete local cryptographic authority, suggesting that the framework does not impose prohibitive technical overhead.

Finally, we proposed the **Protective Legitimacy Score (PLS)** as a candidate metric for comparative evaluation and potential certification. The PLS is presented as a starting point for discussion; its weights and thresholds are provisional and require validation through empirical studies and regulatory consultation.

9.2 Limitations

Several limitations merit acknowledgment. The PLS has not been empirically validated; inter-rater reliability has not been established, and the score does not capture all possible vulnerabilities (e.g., physical side-channel attacks). The reference implementation, while demonstrating feasibility, has not undergone independent audit or large-scale user testing. State detection mechanisms (distinguishing S_2 from S_3) remain an open research problem, and the framework's assumptions about device integrity may not hold in all threat models. The framework prioritizes certain values (safety, sovereignty) over others (convenience, engagement); this hierarchy may not be appropriate for all contexts and should be subject to domain-specific adaptation.

9.3 Open Questions

The framework raises questions that require community engagement. How should systems balance the competing demands of usability and security in the Lost Key Paradox? Can biometric stress detection be implemented locally without creating new surveillance vectors? What regulatory mechanisms would be appropriate for certifying protective architectures? How should the PLS weights be adjusted for different application domains? These questions are not weaknesses of the framework but rather define a research agenda for the field.

9.4 A Research Program, Not a Manifesto

This paper does not claim to have solved the problem of designing for vulnerability. It offers a framework—a set of concepts, formalisms, and principles—that we hope will enable systematic

investigation of how software architecture interacts with human safety. Whether Protective Computing becomes a recognized subdiscipline depends not on the persuasiveness of this single paper, but on whether the research community finds the framework useful for generating new questions, new experiments, and new designs.

The history of consumer computing optimized for convenience. The conditions of the present—climate instability, systemic fragility, institutional erosion—suggest that safety can no longer be treated as a secondary concern. We invite the community to refine, critique, and extend the ideas presented here. The goal is not to declare a new orthodoxy, but to open a line of inquiry that has been neglected for too long: how to build systems that protect when users cannot protect themselves.

References

- [1] A. Cavoukian. 2009. Privacy by Design: The 7 Foundational Principles. Information and Privacy Commissioner of Ontario.
- [2] R. Chatfield, R. Hegarty, and K. Levy. 2023. Coercive Control and Digital Design: A Framework for Safety. *ACM Transactions on Computer-Human Interaction* (2023).
- [3] N. Dell, N. Matthews, and K. Levy. 2023. Rewriting the Code: Technologists Confronting Domestic Abuse. *Interactions* (2023).
- [4] European Parliament and Council. 2016. General Data Protection Regulation (GDPR). Regulation (EU) 2016/679.
- [5] D. Freed, J. Palmer, D. Minchala, and K. Levy. 2019. A Stalker's Paradise: How Intimate Partner Abusers Exploit Technology. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.
- [6] D. Freed, J. Palmer, D. Minchala, K. Levy, T. Ristenpart, and N. Dell. 2018. Digital Technologies and Intimate Partner Violence: A Qualitative Analysis with Survivors. In *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*.
- [7] B. Friedman and D. G. Hendry. 2019. *Value Sensitive Design: Shaping Technology with Moral Imagination*. MIT Press.
- [8] B. Friedman, P. H. Kahn, and A. Borning. 2006. Value Sensitive Design and Information Systems. In *Human-Computer Interaction in Management Information Systems*. M. E. Sharpe.
- [9] A. Gruntz. 2016. Building Offline-First Applications with CouchDB and PouchDB. *Java Magazine* (2016).
- [10] S. Gürses and J. van Hoboken. 2018. Privacy After the Agile Turn. In *Cambridge Handbook of Consumer Privacy*. Cambridge University Press.
- [11] R. Hegeman. 2022. CRDTs and the Quest for Distributed Consistency. *Queue* (2022).
- [12] E. Hollnagel. 2014. *Safety-I and Safety-II: The Past and Future of Safety Management*. CRC Press.
- [13] E. Hollnagel, D. D. Woods, and N. Leveson. 2006. *Resilience Engineering: Concepts and Precepts*. Ashgate.
- [14] J. Hughes. 2015. *Offline First: Building Apps That Work Without the Internet*. O'Reilly Media.
- [15] Internet Engineering Task Force. 1997. RFC 2119: Key words for use in RFCs to Indicate Requirement Levels. RFC 2119.
- [16] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan. 2019. Local-First Software: You Own Your Data, in Spite of the Cloud. In *Proceedings of the ACM Symposium on User Interface Software and Technology (UIST)*.
- [17] A. W. Kushniruk. 2001. Analysis of Complex Decision-Making Processes in Health Care: Cognitive Engineering Methods. *International Journal of Medical Informatics* (2001).
- [18] Barry Leiba. 2017. RFC 8174: Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. RFC 8174. doi:10.17487/RFC8174
- [19] K. Levy and B. Schneier. 2020. Privacy Threats in Intimate Relationships. *Journal of Cybersecurity* (2020).
- [20] N. Matthews, C. N. Tschider, and K. Levy. 2021. Digital Threats to Democracy: Coercive Control and Surveillance by Design. *Journal of Law and Innovation* (2021).
- [21] J. A. Obar and A. Oeldorf-Hirsch. 2020. The Biggest Lie on the Internet: Ignoring the Privacy Policies and Terms of Service Policies of Social Networking Services. *Information, Communication & Society* (2020).
- [22] P. O'Beirne. 2018. Offline First: The New Front in Mobile Development. *IEEE Software* (2018).
- [23] V. L. Patel, T. G. Kannampallil, and D. R. Kaufman. 2017. *Cognitive Informatics in Health and Biomedicine*. Springer.
- [24] J. Reason. 1990. *Human Error*. Cambridge University Press.
- [25] N. Richards and W. Hartzog. 2023. *Privacy's Trust: The Path to Digital Dignity*. Cambridge University Press.
- [26] I. S. Rubinstein and N. Good. 2020. Privacy by Design: A Counterfactual Analysis of Google and Facebook. *NYU Law Review* (2020).
- [27] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. 2011. *A Comprehensive Study of Convergent and Commutative Replicated Data Types*. Research Report. INRIA.

[28] K. Shilton. 2013. Engaging Values Despite Neutrality: Challenges and Approaches to Values Reflection during the Design of Internet Infrastructure. *Science, Technology, & Human Values* (2013).

[29] L. Stark. 2020. Algorithmic Personality Testing and the Illusion of Control. *Georgetown Law Technology Review* (2020).

[30] E. Tsukayama, J. Simpson, and K. Levy. 2022. The Privacy-Security-Surveillance Complex. *Stanford Law Review Online* (2022).

[31] A. F. van Wynsberghe. 2015. *Healthcare Robots: Ethics, Design and Implementation*. Routledge.

[32] E. L. Wiener and D. C. Nagel. 1988. *Human Factors in Aviation*. Academic Press.

[33] D. D. Woods. 2015. Four Concepts for Resilience and the Implications for the Future of Resilience Engineering. *Reliability Engineering & System Safety* (2015).

[34] D. D. Woods and E. Hollnagel. 2006. *Joint Cognitive Systems: Patterns in Cognitive Systems Engineering*. CRC Press.

A Notation Reference

Table 7. Notation Reference

Symbol	Meaning
H	Total Harm (function of D, V, I)
D	System Dependency
V	User Vulnerability
I	Outcome Irreversibility
S_0-S_3	Vulnerability States
P_i	Institutional Power
P_u	User Sovereignty
ΔP	Sovereignty Gap ($P_i - P_u$)
R	Asymmetry Risk
S	Data Sensitivity
PLS	Protective Legitimacy Score
RQ	Reversibility Quotient
ER	Exposure Ratio
LAI	Local Authority Index
CLF	Cognitive Load Factor
ΔS	Sovereignty Delta

B Audit Framework Dimensions (Preliminary)

A formal Protective Audit would evaluate:

- (1) **Source Code Analysis**
 - Verification of embedded database authority (Principle 3)
 - Confirmation of client-side encryption implementation (Principle 2)
 - Inspection of deletion/restoration logic (Principle 1)
- (2) **State Transition Testing**
 - Forced degradation of network, power, and authentication infrastructure
 - Observation of system behavior under simulated S_2 and S_3 conditions
- (3) **Exportability Verification**
 - Attempted offline data export under zero-network conditions
 - Validation of non-proprietary format compliance